

Course Code: **IDC 222**



Scientific Computing & Data Visualization

Reji J

School of Data Science

IISER Thiruvananthapuram

Set



Associative Container

- Associative containers store data in a **sorted order** and provide **fast search, insertion, and deletion**.

Types of Associative Containers

Container	Description
Set	Collection of unique elements sorted by their values.
Multiset	Collection of elements sorted by values, duplicates allowed .
Map	Collection of key-value pairs sorted by keys . Keys are unique .
Multimap	Collection of key-value pairs sorted by keys . Duplicates allowed .

- All containers maintain **automatic sorting**.
- Use **set/map** when **uniqueness is required**.
- Use **multiset/multimap** when **duplicates are allowed**.



Set

- A **set** is an STL container that stores **unique elements** of the same type in **sorted order**.
- Each element is **unique**, so the element itself acts as a **key**.

Set Properties

1. **Unique Elements:** No duplicates allowed.
2. **Immutable:**
 - We can **insert** or **erase** elements
 - But we **cannot modify** the value of an existing element.
3. **Sorted Order**
 - By default, sets are sorted in **ascending order**, but we have the option to change that.
4. **Associative Container**
 - Elements are accessed using **keys**, not by **indexes**.
 - Unlike arrays, there is **no direct position-based access**.



Set - Header File & Syntax

Using Set in C++

To use sets, include the header file:

```
#include <set>
```

Set Declaration

Syntax: `set<T> set_name;`

Example: `set<int> numbers;`
 `set<float> marks;`
 `set<string> names;`

- `<T>` specifies the data type of the vector.
- It can be: `int char float double`



Set Initialization

Syntax: `set<data_type> set_name = {key1, key2, key3, ...};`

- `key1, key2, key3, . . .` → Elements to store in the set

Example:

```
// Initialize set with elements
set<int> s1 = {5, 3, 8, 1, 3};
set<int> s2 = {'d', 'a', 'c', 'b', 'a'};
```

```
// Create an empty set
set<int> s3;
```

- **Duplicates removed automatically**
- Stored in sorted order: **1 3 5 8**
- Stored in sorted order: **a b c d**
- Cannot access elements by index []



Sample Program

```
#include <iostream>
#include <set>
using namespace std;

int main()
{
    set<int> numberSet = {5, 3, 8, 1, 3};

    for(int val : numberSet)
    {
        cout << val << " ";
    }

    return 0;
}
```

Output:

1 3 5 8

- Set automatically sorts elements in **ascending order**.
- Repeated values (like **3**) are **ignored** automatically.

Note:

- Cannot use **[] operator** with sets.
numberSet[4] = 50; ❌ // Not allowed!
- Set elements are **keys** themselves and are **immutable**.



Program

```
#include <iostream>
#include <set>
using namespace std;

int main()
{
    set<char> charSet = {'d', 'a', 'c', 'b', 'a'};
    cout << "Character set: ";
    for(char ch : charSet)
    {
        cout << ch << " ";
    }
    cout << endl;

    set<string> stringSet = {"apple", "orange", "banana", "apple"};
    cout << "String set: ";
    for(string str : stringSet)
    {
        cout << str << " ";
    }
    cout << endl;

    return 0;
}
```

Output:

Character set: a b c d

String set: apple banana orange



Sort a Set in Descending Order

- By default, a set stores elements in **ascending order**
- To sort elements in **descending order**, use a **comparison function**

Syntax: `set< int , greater<int> > set_name;`

Example: `set< int , greater<int> > my_set = {5, 3, 8, 1, 3};`

Program:

```
#include <iostream>
#include <set>
using namespace std;

int main()
{
    set<int, greater<int>> my_set = {5, 3, 8, 1, 3};
    for(int val : my_set)
    {
        cout << val << " ";
    }
    return 0;
}
```

OUTPUT:

8 5 3 1

set now returns the elements sorted in descending order.

Important Functions in Set

- In C++, the **set** class provides various **functions** to perform various **operations** in a set.

Function	Purpose
<code>insert(x)</code>	Insert an element into the set
<code>erase(x)</code>	Remove an element from the set
<code>find(x)</code>	Search for an element
<code>empty()</code>	Checks if vector is empty
<code>clear()</code>	Removes all elements
<code>size()</code>	Returns number of elements in the map



Basic Operations on a Set

1. Inserting Elements

- Elements are added using the **insert()** function
- Duplicate values are ignored and each element is stored only once
- Elements are automatically arranged in sorted order

Program:

```
int main()
{
    set<int> my_set = {10, 40, 20};

    // Insert elements into set
    my_set.insert(50);
    my_set.insert(30);
    my_set.insert(50);

    for(int i : my_set)
    {
        cout << i << " ";
    }
    cout << endl;
    return 0;
}
```

Syntax:

```
setname.insert(value);
```

OUTPUT:

```
10 20 30 40 50
```



2. Accessing Elements

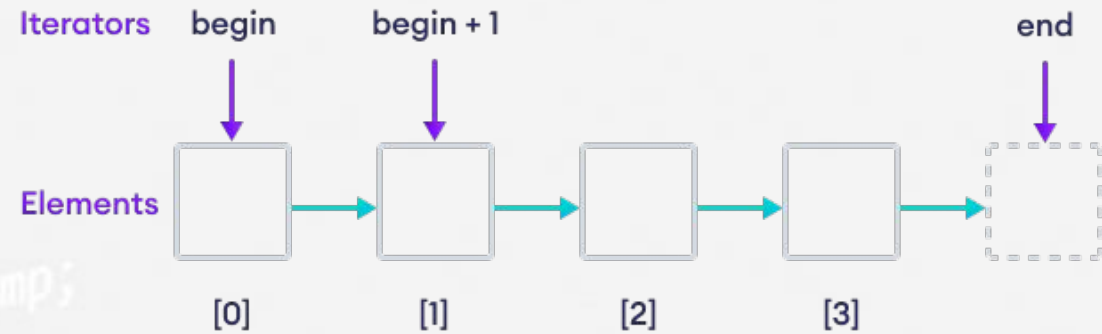
- We can't access elements of a set by index like vector.
- In set, we have to use **iterator** to access the elements by position.

Program:

```
int main()
{
    set<int> s = {50, 10, 40, 30, 20};

    // Create a set iterator called it
    set<int>::iterator it;

    for(it = s.begin(); it != s.end(); it++)
    {
        cout << *it << " ";
    }
    cout << endl;
    return 0;
}
```



OUTPUT:

10 20 30 40 50



Accessing Elements in a Set using `next()`

- `begin()` returns an iterator to the first element of the set.
- `next()` is used to move the iterator forward by a given number of positions.
- Elements are accessed using the dereference operator (*).

Program:

```
#include <iostream>
#include <set>
#include <iterator>
using namespace std;

int main()
{
    set<int> s = {10, 40, 20, 30, 50};

    set<int>::iterator it1 = s.begin();

    auto it2 = next(it1, 1);    // move 1 position
    auto it3 = next(it1, 2);    // move 2 positions

    cout << *it1 << " " << *it2 << " " << *it3 << endl;

    return 0;
}
```

OUTPUT:

10 20 30



3. Updating Elements

- Elements in a set **cannot** be updated.
- Once inserted, values are **immutable**
- Direct modification is **not allowed**
- We **cannot change** the value of elements once they are stored in the set.
- To update an element in a set, **first erase it**, then **insert the new value**.

Not Allowed ❌

```
*it = 10;
```

Correct Way (Delete & Insert) ✅

```
my_set.erase(5);  
my_set.insert(10);
```



4. Traversing

- Set elements can be easily traversed by using:
 - Range-based for loop
 - Iterators with `begin()` and `end()`

Program:

```
set<int> s = {50, 10, 40, 30, 20};

// Traversing using Range-Based for Loop
for (int val : s)
{
    cout << val << " ";
}

// Traversing using iterators
for (auto it = s.begin(); it != s.end(); ++it)
{
    cout << *it << " ";
}
```

OUTPUT:

10 20 30 40 50



5. Finding Elements

- set provides fast searching using the `find()` function
- `find()` searches by value (key).
- This function Returns: Iterator to the element → if found
`end()` iterator → if not found

Syntax:

```
iterator = setname.find(value);
```

Program:

```
int main()
{
    set<int> s = {1, 4, 2, 3, 5};
    set<int>::iterator it;

    it = s.find(5);    // Finding element 5
    if(it != s.end())
    {
        cout << *it << " is in the set" << endl;
    }
    else
    {
        cout << "Element not found!" << endl;
    }
    return 0;
}
```

OUTPUT:-

5



Why do we check `it != s.end()` ?

- `find()` does not return true or false
- It returns an **iterator**

CASE 1: Element is found

```
it = s.find(5);
```

- `it` points to the element `5`
- `it` is NOT equal to `s.end()`
- So this condition becomes:

```
it != s.end() // true
```

CASE 2: Element is not found

```
it = s.find(9);
```

- `find()` cannot locate the element
- returns `s.end()`
- So

```
it != s.end() // false
```



6. Deleting Elements

- Elements in a set are removed using the **erase()** function.
- set allows deletion **directly** using the value

Syntax: `setname.erase(value);`

Program:

```
#include <iostream>
#include <set>
using namespace std;

int main()
{
    set<int> my_set = {1, 4, 2, 3, 5};

    my_set.erase(5);    // Delete element by value

    for(int i : my_set)
    {
        cout << i << " ";
    }
    return 0;
}
```

OUTPUT:-

1 2 3 4



`empty()`, `clear()` and `size()`

- To remove all elements from a set, you can use the `clear()` function:

Example:

```
set<string> names = {"Adam", "Bob", "David"};

// Remove all elements
names .clear();
```

- To find out how many elements a set has, use the `size()` function:

Example:

```
set<string> names = {"Adam", "Bob", "David"};

cout << names.size(); // Outputs 3
```

- Use the `empty()` function to find out if a set is **empty or not**.
- The `empty()` function **returns 1 (true)** if the set is **empty** and **0 (false)** otherwise:

Example:

```
set<string> names;

cout << names.empty(); // Outputs 1 (Set is empty)
```



Program: Set Operations

```
#include <iostream>
#include <set>
using namespace std;

int main()
{
    set<int> numbers;
    int n, value;

    cout << "Enter the number of elements: ";
    cin >> n;

    cout << "Enter elements:\n";
    for (int i = 0; i < n; i++)
    {
        cin >> value;
        numbers.insert(value);
    }

    cout << "Set elements (sorted automatically): ";
    for (int num : numbers)
    {
        cout << num << " ";
    }
    cout << endl;
```

```
// Checking if an element exists
int search = 5;

if (numbers.find(search) != numbers.end())
{
    cout << search << " is found!" << endl;
}
else
{
    cout << search << " is not found!" << endl;
}

numbers.erase(10);
cout << "After deletion, set elements: ";
for (int num : numbers)
{
    cout << num << " ";
}
cout << endl;
cout << "Size of the set: " << numbers.size();
return 0;
```



String using Iterator

```
#include <iostream>
#include <string>
using namespace std;

int main()
{
    string str = "hello";
    cout << "Characters in string: " << endl;
    for(auto it = str.begin(); it != str.end(); it++)
    {
        cout << *it << "\t";
    }
    return 0;
}
```

- A **string** supports iterators
- **begin()** → first character
- **end()** → position after last character

OUTPUT:-

Characters in string:
h e l l o



String reversal

- Reverse iterators traverse the string **from end to beginning**

```
#include <iostream>
#include <string>
using namespace std;

int main()
{
    string str;
    cout << "Enter a string: ";
    cin >> ws; // clear whitespace
    getline(cin, str);
    cout << "Reversed string: ";
    for(auto it = str.rbegin(); it != str.rend(); it++)
    {
        cout << *it;
    }
    return 0;
}
```



String and Set

- A **set** can be used to store unique characters from a string

```
#include <iostream>
#include <set>
#include <string>
using namespace std;

int main()
{
    string str = "banana";

    // Store unique characters from string
    set<char> uniqueChars(str.begin(), str.end());

    cout << "Unique characters: ";
    for(char ch : uniqueChars)
    {
        cout << ch << "\t";
    }
    return 0;
}
```

OUTPUT:-

Unique characters:

a b n





Exercise : Find Common Characters in Two Strings Using Set

Write a C++ program to find and display the common characters present in two strings using STL set.

- Read two strings from the user
- Store characters of each string in a set
- Find and display common characters
- Output should contain unique characters only

OUTPUT FORMAT:

First string : orange

Second string : mango

Common characters: a g n o

Find Common Characters in Two Strings

```
#include <iostream>
#include <set>
using namespace std;

void findCommonCharacters(string str1, string str2)
{
    set<char> set1(str1.begin(), str1.end());
    set<char> set2(str2.begin(), str2.end());
    set<char> common;

    for (char ch : set1)
    {
        if (set2.find(ch) != set2.end())
        {
            common.insert(ch);
        }
    }

    cout << "Common characters: ";
    for (char ch : common)
    {
        cout << ch << " ";
    }
    cout << endl;
}
```

```
int main()
{
    string str1, str2;
    cout << "First string : ";
    cin>>ws;
    getline(cin, str1);
    cout << "Second string : ";
    cin>>ws;
    getline(cin, str2);
    findCommonCharacters(str1, str2);

    return 0;
}
```





Exercise : Find Union and Intersection Using Set and Vector

Write a C++ program to find the Union and Intersection of two integer collections using STL vector and set.

- Store elements in two `vector<int>`
- Use `set<int>` to:
 - Find **Union**
 - Find **Intersection**
- Display the results

INPUT:

{1, 3, 4, 5, 7}

{2, 3, 5, 6}

OUTPUT:

Union: 1 2 3 4 5 6 7

Intersection: 3 5

Find the Union and Intersection of Two Vectors

```
#include <iostream>
#include <set>
#include <vector>
using namespace std;

void printSet(set<int> s)
{
    for(int num : s)
        cout << num << " ";
    cout << endl;
}

int main()
{
    vector<int> v1 = {1, 3, 4, 5, 7};
    vector<int> v2 = {2, 3, 5, 6};

    set<int> unionSet, intersectionSet;

    // Insert elements of first vector
    for(int num : v1)
    {
        unionSet.insert(num);
    }

    // Insert elements of second vector
    for(int num : v2)
    {
        unionSet.insert(num);
    }

    // Convert first vector to set for intersection
    set<int> set1(v1.begin(), v1.end());

    // Find common elements
    for(int num : v2)
    {
        if(set1.find(num) != set1.end())
        {
            intersectionSet.insert(num);
        }
    }

    cout << "Union: ";
    printSet(unionSet);

    cout << "Intersection: ";
    printSet(intersectionSet);

    return 0;
}
```



STL Containers

Sequence Containers:

Array:



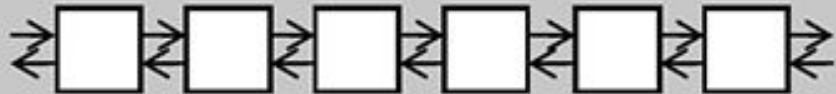
Vector:



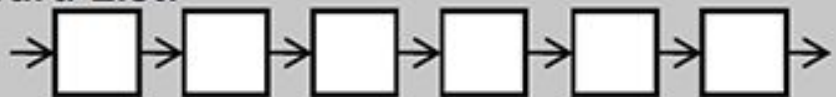
Deque:



List:

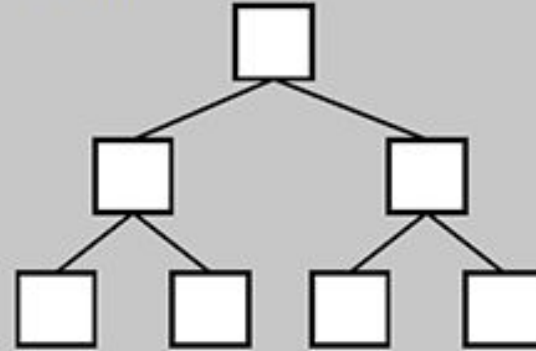


Forward-List:

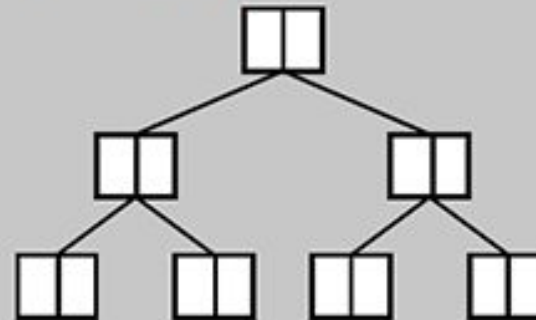


Associative Containers:

Set/Multiset:

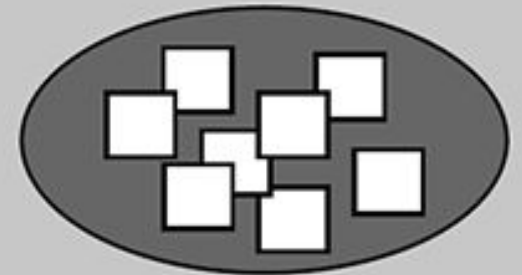


Map/Multimap:

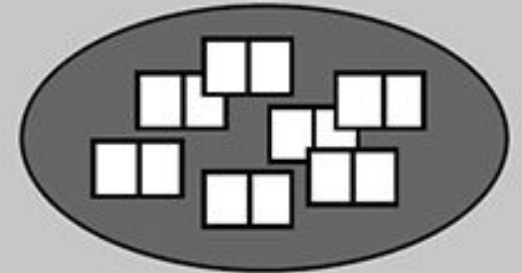


Unordered Containers:

Unordered Set/Multiset:



Unordered Map/Multimap:

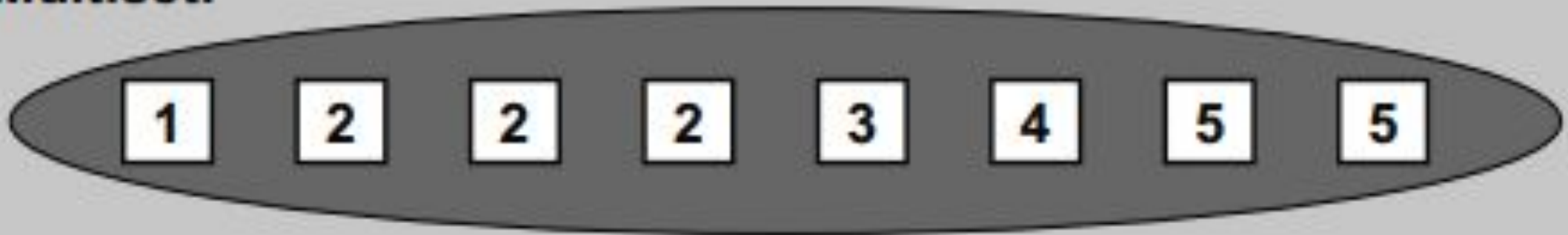


Set & MultiSet

Set:



Multiset:

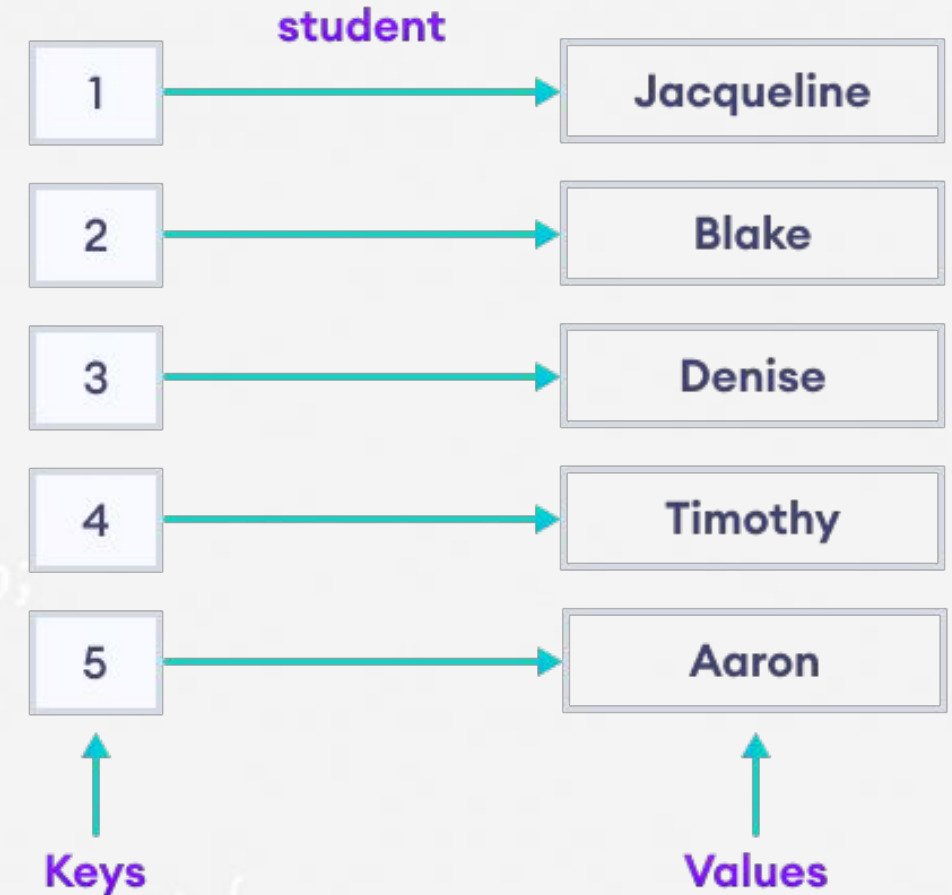


Map



Map

- map is an **associative container**
- It stores data in **key-value pairs**
- **Keys are unique**
- **Values can be duplicate**
- Data is stored in **sorted order (by key)**



Syntax: `map<key_type, value_type> map_name = {{key1, value1}, {key2, value2}, ... };`

Example: `map<int, string> student = {{1, "Adam"}, {2, "Ilesh"}, {3, "John"}};`



Map - Header File & Syntax

Using Map in C++

To use maps, include the header file:

```
#include <map>
```

Map Declaration

Syntax: `map<key_type, value_type> map_name;`

Example: `map<int, string> student;`

- `<key_type>` → Data type of the key
- `<value_type>` → Data type of the value
- `map_name` → Name of the map

- Key → `int` (Student ID)
- Value → `string` (Student Name)



Map Initialization

Syntax: `map<key_type, value_type> map_name = {{key1, value1}, {key2, value2}, ... };`

- `key1, key2, ...` → Keys to be stored in the map
- `value1, value2, ...` → Values associated with the keys
- Each element is stored as a **key-value pair**

Example: `map<int, string> student = {{1, "Adam"}, {2, "Ilesh"}, {3, "John"}};`

- Keys must be **unique**
- Elements are **automatically sorted by key**
- Values can be **duplicated**



Sample Program

```
#include <iostream>
#include <map>
using namespace std;

int main()
{
    map<int, string> student = { {101, "Adam"}, {102, "Ilesh"}, {103, "John"} };

    for(auto it = student.begin(); it != student.end(); it++)
    {
        cout << it->first << " : " << it->second << endl;
    }

    return 0;
}
```

Note:

`it->first` → key

`it->second` → value

OUTPUT :

101 : Adam

102 : Ilesh

103 : John



Important Functions in Map

- In C++, the **map** class provides various **functions** to perform various **operations** in a map.

Function	Purpose
<code>insert({k, v})</code>	Insert a key-value pair into the map
<code>erase(k)</code>	Remove element with key k
<code>find(k)</code>	Returns iterator to key k if found, else end()
<code>empty()</code>	Checks if vector is empty
<code>clear()</code>	Removes all elements
<code>size()</code>	Returns number of elements in the map



Sample Program

```
#include <iostream>
#include <map>
using namespace std;

int main()
{
    map<int, string> lang = {{1, "C"}, {2, "C++"}, {3, "Python"}};

    for (auto i : lang)
    {
        cout << i.first << " " << i.second << endl;
    }
    return 0;
}
```

Note:

first → key

second → value

OUTPUT:

1 C

2 C++

3 Python



Basic Operations on a Map

1. Inserting elements using `insert()`

- Inserts a **new key-value pair**
- If the **key already exists**, insertion is **ignored**
- We cannot specify a position for insertion

Syntax:

```
map_name.insert({key, value});
```

Program:

```
#include <iostream>
#include <map>
using namespace std;

int main()
{
    map<int, string> lang = {{2, "C++"}, {3, "Python"}};

    lang.insert({1, "C"});
    lang.insert({4, "Java"});

    for (auto i : lang)
    {
        cout << i.first << " " << i.second << endl;
    }
    return 0;
}
```

OUTPUT:

```
1 C
2 C++
3 Python
4 Java
```

2. Accessing Elements

1) Using `[]` Operator

- Returns the value if the **key exists**
- If the **key does not exist**, it **creates a new element**
- New element gets a **default value**

```
lang[1] // Access existing key
lang[5] // Creates key 5 with default value ""
```

2) Using `at()` function

- Returns the value if the **key exists**
- **Does NOT create** a new element
- Safer for accessing elements

Program:

```
int main()
{
    map<int, string> lang = {{1, "C++"}, {2, "Python"}};
    // Accessing elements
    cout << lang[1] << endl;    // Using []
    cout << lang.at(2) << endl; // Using at()
    return 0;
}
```

OUTPUT :

C++

Python

3. Updating Elements

- **Keys cannot be changed** once they are stored in a map
- **Only the associated value can be updated**
- Updating is done by **accessing the key** and assigning a new value

Ways to Update Values

- Using **[] operator**
- Using **at()** (safer)

Program:

```
int main()
{
    map<int, string> lang = { {1, "C"}, {2, "C++"}, {3, "Python"} };

    // Updating values

    lang[1] = "JavaScript";    // Using []
    lang.at(2) = "R";          // Using at()

    for (auto i : lang)
    {
        cout << i.first << " " << i.second << endl;
    }
    return 0;
}
```

OUTPUT:

```
1 JavaScript
2 R
3 Python
```

4. Traversing

- Map elements can be easily traversed by using:
 - Range-based for loop
 - Iterators with `begin()` and `end()`

Program:

```
map<int, string> lang = {{1, "C"}, {2, "C++"}, {3, "Python"}};

// Traversing using iterators
for (auto it = lang.begin(); it != lang.end(); ++it)
{
    cout << it->first << " " << it->second << endl;
}

// Traversing using Range-Based for Loop
for (auto i : lang)
{
    cout << i.first << " " << i.second << endl;
}
```

Note:

first → key

second → value

it->first → key

it->second → value

OUTPUT:

1 C

2 C++

3 Python



5. Finding Elements

- Maps provide fast searching using the **find()** function.
 - **find(key)** searches for an element using its key
 - If the key is found, it returns an **iterator** to that element
 - If the key is **not found**, it returns **end()**

Syntax:

```
map_name.find(key);
```

Program:

```
int main()
{
    map<int, string> lang = {{1, "C"}, {2, "C++"}, {3, "Python"}};
    // Finding element with key 2
    auto it = lang.find(2);
    if (it != lang.end())
    {
        cout << it->first << " " << it->second;
    }
    else
    {
        cout << "Key not Found!" << endl;
    }
    return 0;
}
```

OUTPUT:

2 C++



6. Deleting Elements

- Map elements can be removed using the **erase()** function by passing the key.

Syntax: `map_name.erase(key);`

```
#include <iostream>
#include <map>
using namespace std;

int main()
{
    map<int, string> lang = { {1, "C"}, {2, "C++"}, {3, "Python"} };

    // Deleting element with key 2
    lang.erase(2);

    for (auto i : lang)
    {
        cout << i.first << " " << i.second << endl;
    }
    return 0;
}
```

OUTPUT:

1 C

3 Python





Exercise : Student Records Using Map

Write a C++ program to store student roll numbers and names using a map.

- Read the number of students n
- Read each student's roll number and name
- Store the data in a map
- Display all student records sorted by roll no

OUTPUT FORMAT:

Enter the number of students: 3

Enter Roll Number and Name:

3 Rahul

1 Anu

2 Kiran

Student Records

Roll No: 1, Name: Anu

Roll No: 2, Name: Kiran

Roll No: 3, Name: Rahul

```

#include <iostream>
#include <map>
using namespace std;

int main()
{
    map<int, string> students;
    int n, roll;
    string name;

    cout << "Enter the number of students: ";
    cin >> n;

    cout << "Enter Roll Number and Name:\n";
    for (int i = 0; i < n; i++)
    {
        cin >> roll;
        cin >> ws;           // To clear whitespace
        getline(cin, name);   // To read full name

        students.insert({roll, name});
    }

    cout << "\nStudent Records (Sorted by Roll Number):\n";
    for (auto pair : students)
    {
        cout << "Roll No: " << pair.first << ", Name: " << pair.second << endl;
    }
    return 0;
}

```





Exercise (Continuation) : Implement Search Functionality

Extend the previous program to search for a student by roll number using a map.

- Accept a roll number from the user
- Use the `find()` function to search the map
- If the roll number exists, display the student details
- Otherwise, display “Student not found”

```

#include <iostream>
#include <map>
using namespace std;

int main()
{
    map<int, string> students;
    int n, roll;
    string name;

    cout << "Enter the number of students: ";
    cin >> n;

    cout << "Enter Roll Number and Name:\n";
    for (int i = 0; i < n; i++)
    {
        cin >> roll;
        cin >> ws;           // To clear whitespace
        getline(cin, name);   // To read full name

        students.insert({roll, name});
    }

    cout << "\nStudent Records (Sorted by Roll Number):\n";
    for (auto pair : students)
    {
        cout << "Roll No: " << pair.first << ", Name: " << pair.second << endl;
    }
}

```

```

// Search functionality
cout << "\nEnter Roll Number to search: ";
cin >> roll;

auto it = students.find(roll);

if (it != students.end())
{
    cout << "Student found: "
         << it->first << " "
         << it->second << endl;
}
else
{
    cout << "Student not found!" << endl;
}

return 0;
}

```



Exercise: Student Management System using Map

Write a C++ program using map to manage student records with a menu-driven approach.

The program should allow the user to:

1. Add a student (ID and Name)
2. View all students
3. Update a student's name using ID
4. Delete a student using ID
5. Exit the program

Program: Student Database

```
#include <iostream>
#include <map>
using namespace std;

int main()
{
    map<int, string> students;
    int choice, id;
    string name;

    do {
        cout << "\n1. Add Student\n2. View Students\n3. Update Student\n4. Delete Student\n5. Exit\n";
        cout << "Enter choice: ";
        cin >> choice;

        if (choice == 1)
        {
            // Add Student
            cout << "Enter Student ID: ";
            cin >> id;
            cout << "Enter Name: ";
            cin >> name;
            students.insert({id, name}); // Insert
            cout << "Student added successfully!\n";
        }
    }
```



```

else if (choice == 2)
{
    // View Students
    if (students.empty())
    {
        cout << "No students found!\n";
    } else
    {
        cout << "Student List:\n";
        for (auto entry : students)
        {
            cout << "ID: " << entry.first << ", Name: " << entry.second << endl;
        }
    }
}
else if (choice == 3)
{
    // Update Student
    cout << "Enter Student ID to update: ";
    cin >> id;
    if (students.find(id) != students.end())
    {
        cout << "Enter new Name: ";
        cin >> name;
        students.at(id) = name; // Update
        cout << "Student updated successfully!\n";
    } else
    {
        cout << "Student ID not found!\n";
    }
}
}

```



```

else if (choice == 4)
{
    // Delete Student
    cout << "Enter Student ID to delete: ";
    cin >> id;
    if (students.erase(id))
    {
        cout << "Student deleted successfully!\n";
    } else
    {
        cout << "Student ID not found!\n";
    }
}
else if (choice == 5)
{
    cout << "Exiting...\n";
}
else
{
    cout << "Invalid choice! Try again.\n";
}

} while (choice != 5);

return 0;
}

```



MultiSet



Multi Set

- Multiset is an associative container similar to the set, but it can store multiple elements with same value.
- In other words, **duplicate values** are **allowed**.

Set Properties

- **Associative:** Elements are referenced by their key and not by their absolute position in the container.
- **Sorted:** Elements are stored in a sorted manner.
- **Equivalent keys:** Multiple elements in the container can have equivalent keys.
- **Immutable:** The value of elements can't be modified after they have been stored in the multiset.



MultiSet Declaration

- To implement a multiset in C++, we must include the `<set>` header file in our program.

```
#include <set>
```

- We can declare a set in C++ using the following syntax:

Syntax:

```
multiset<data_type> multiset_name = {key1, key2, key3, ...};
```

- Here, `key1, key2, key3, ...` - **key/value** to be stored in the set.

Example:

```
// initialize set with elements
multiset<int> my_multiset1 = {5, 3, 8, 1, 3};

// create an empty set
multiset<int> my_multiset2;
```



Sample Program

```
#include <iostream>
#include <set>
using namespace std;

int main()
{
    multiset<int> my_multiset = {5, 3, 8, 1, 3};

    for(int val : my_multiset)
    {
        cout << val << " ";
    }

    return 0;
}
```

Output:

1 3 3 5 8

As you can see, the multiset returned the values in a sorted manner and included the **duplicate** instances of 3.



MultiMap



MultiMap

- **Multimap** is an associative container similar to map, but it can have **multiple elements with same keys**.

MultiMap Declaration

- To use **Multimaps**, we must **include** the **map** header file:
- We can declare a **multimap** using the following syntax:

```
#include <map>
```

Syntax:

```
multimap<key_type, value_type> multimap_name = {{key1, value1},{key2, value2}, ...};
```

Example:

```
multimap<int,string> lang = {{1, "C"},{2, "C++"}, {1, "Python"}, {2, "R"}};
```



Sample Program

```
#include <iostream>
#include <map>
using namespace std;

int main()
{
    multimap<int, string> lang = {{1, "C"}, {2, "C++"}, {1, "Python"}, {2, "R"}};

    for (auto i : lang)
    {
        cout << i.first << " " << i.second << endl;
    }
    return 0;
}
```

OUTPUT :

```
1 C
1 Python
2 C++
2 R
```

